

Egor Uzhanin

🔗 github.com/gearonixx | 🌐 gearonixx.vercel.app | @ gearonixx@gmail.com | Telegram: @gearonixx | X: @kasper_0q

SKILLS

Programming: C++ (11/14/17/20), C, Rust, Python, SQL, Bash

Technologies: STL, Boost, libevent, gRPC, Protobuf, FlatBuffers, ActiveMQ, Apache Cassandra, ClickHouse, NoSQL/SQL databases, ZeroMQ

Systems: Multi-threading, Lock-free Data Structures, Memory-mapped I/O, SIMD, Kernel Bypass (DPDK), TCP/UDP Networking, IPC, Shared Memory

Domains: High-Frequency Trading, Real-Time Bidding, Algorithmic Trading, Distributed Systems, Blockchain

Methodologies: Architectural Design, R&D, High-Performance Computing, Agile, Scrum, Test-Driven Development

Tools: gdb, Valgrind, perf, strace, AddressSanitizer, ThreadSanitizer, Benchmarking, CMake, Conan, Bazel, Jenkins, GitLab CI/CD

Infrastructure: Git, GitLab, Docker, Docker Compose, Kubernetes, Linux (kernel tuning, cgroups, namespaces)

Spoken: English (B2), German (B1)

EXPERIENCE

Optiver

Amsterdam, Netherlands

Senior Software Engineer (C++), Low-Latency Trading Systems

Dec 2022 – Present, Full-time

- Lead the design and development of server-side modules for an electronic trading product deployed on major stock exchanges, ensuring sub-microsecond deterministic execution paths.
- Architected and implemented real-time market data and order execution connectors for Eurex, XETRA, HKEX and KRX using native exchange binary protocols over TCP/multicast UDP.
- Migrated the legacy trading server from Solaris SPARC to Linux x86-64, re-engineering critical hot paths with lock-free queues, memory-mapped ring buffers and CPU affinity pinning.
- Reduced median order-to-ack latency by 40% through systematic profiling with perf, FlameGraphs and custom micro-benchmarks, eliminating allocations and cache misses on the critical path.
- Designed a zero-copy serialisation layer using FlatBuffers to replace legacy Protobuf encoding, cutting message serialisation overhead by 3x.
- Implemented a fault-tolerant failover mechanism with deterministic replay from an in-house journal, achieving RTO <2s across active-passive pairs.
- Provided tier-3 production support, diagnosing complex concurrency bugs and network-level issues using gdb, strace, tcpdump and custom instrumentation.

Citadel Securities

London, United Kingdom

Senior Software Engineer (C/C++), Real-Time Bidding Platform

Jan 2020 – Dec 2022, Full-time

- Designed and built the core auction engine of a high-load real-time bidding (RTB) platform in C/C++ using libevent, processing 500K+ bid requests per second with P99 latency under 5ms.
- Implemented a custom memory pool allocator and arena-based allocation strategy, reducing heap fragmentation and GC pauses in the mixed C++/Java codebase.
- Developed a multi-threaded event-driven pipeline supporting parallel real-time auctions with work-stealing scheduling across NUMA nodes.
- Integrated ClickHouse as the analytics backend, designing a batched async ingestion pipeline that sustained 2M+ events/sec with columnar compression.
- Built a low-latency feature store backed by a custom hash map with open addressing and SIMD-accelerated probing for real-time bid enrichment.
- Conducted systematic performance tuning using Valgrind, AddressSanitizer and ThreadSanitizer, eliminating data races and reducing memory usage by 35%.
- Collaborated with ML researchers to integrate model inference (ONNX Runtime) into the bid evaluation path without violating latency SLAs.

Flow Traders

Amsterdam, Netherlands

Software Engineer (C++), Algorithmic Trading Systems

Mar 2018 – Jan 2020, Full-time

- Contributed to the architecture of a distributed algorithmic trading platform handling 100K+ orders/sec across multiple asset classes in a client-server topology.

- Designed and implemented the order management and execution core in C++17, using template metaprogramming and compile-time polymorphism to eliminate virtual dispatch overhead.
- Developed and maintained FIX and native binary connectors to exchanges and liquidity providers (Bitfinex, Binance, Huobi, OKX, Poloniex), handling connection management, sequencing and recovery.
- Built a lock-free, single-producer/multi-consumer event bus for distributing market data ticks to strategy instances with zero-copy semantics and cache-line-aligned buffers.
- Implemented a risk management layer performing pre-trade checks (position limits, notional caps, rate throttling) on the critical path with <500ns overhead per order.
- Created an automated regression and performance testing framework using Google Test and Google Benchmark, integrated into GitLab CI with nightly latency reports.
- Delivered enterprise-grade trading solutions under strict compliance and audit requirements for a leading proprietary trading firm.

EDUCATION

ITMO University

B.Sc. in Applied Mathematics and Computer Science

Saint Petersburg, Russia

Sep 2019 – Jun 2024

CERTIFICATIONS & COURSES

C++ Development Fundamentals: Brown Belt | [Certificate](#)

- Coursera · May 2022 · Advanced C++ algorithms and data structures.

C++ Development Fundamentals: White Belt | [Certificate](#)

- Coursera · Dec 2024 – Jan 2025 · C++ fundamentals, algorithms and data structures.

Data Structures and Performance | [Certificate](#)

- Coursera · Apr 2023 · Java, algorithms and performance optimisation.

Advanced Data Structures in Java | [Certificate](#)

- Coursera · 2024 · In-depth study of data structures and algorithmic techniques.

Algorithms and Data Structures | [Certificate](#)

- Coursera · 2024 · Core algorithms and data structures for professional software development.

Certified Kubernetes Administrator (CKA) | [Linux Foundation](#)

- Linux Foundation · 2024 · Certified in administering Kubernetes clusters and cloud infrastructure.

Open Source Contributions | [GitHub](#)

- Contributor to *YTsaurus* (distributed storage/processing platform by Yandex, C++), *Bitcoin Core* (C++), *ClickHouse* (columnar DBMS, C++), *Hyperledger Fabric/Firefly* (enterprise blockchain), *TON Blockchain* (C++), *userver* (async C++ framework) and *Telegram Desktop* (C++/Qt). Active forks: *deno_core* (Rust), *haskell-language-server*, *ScyllaDB cpp-rs-driver*.

SUMMARY

C/C++ engineer with 6+ years of experience architecting and delivering high-frequency trading, real-time bidding and algorithmic trading platforms handling millions of events per second.

Deep expertise in low-latency system design: lock-free data structures, custom allocators, zero-copy serialisation, SIMD optimisation and kernel-level tuning. Proven track record building exchange connectors, event-driven pipelines and distributed systems under strict latency and reliability constraints. Active open-source contributor to Bitcoin Core, YTsaurus, ClickHouse, Hyperledger and TON. Comfortable leading technical initiatives, profiling production systems and collaborating in cross-functional teams. Experienced in reading and communicating technical documentation in English.

Security analysis of YTsaurus | [Commit](#)

- Hardened Yandex's distributed big-data platform (C++): guarded query-engine subquery preparation against stack overflow and restricted password-hash access. Maintain a reproducible Dockerized clang-18 / CMake / Conan build environment for the codebase.

Telegram security report — MicroTeX \multirow | [Findings](#)

- Discovered and reported to the Telegram security team a remote, near-zero-click heap out-of-bounds read + bounded OOB write (CWE-125 / CWE-787, CVSS 6.5) in Telegram Desktop's bundled MicroTeX LaTeX renderer (`MatrixAtom::recalculateLine`), reachable via Instant-View math in received messages. Root-caused an unbounded \multirow row-span, built AddressSanitizer proof-of-concepts, produced a symbolized crash backtrace proving the network-receive → layout path, and supplied a two-sided clamp fix. Present in shipped builds since 6.8.3 beta (2026).

Telegram Desktop — crash fix | [Commit](#)

- Fixed a crash triggered when removing pinned chats during folder changes in the C++ desktop client.

Database drivers for Yandex | [userver](#)

- Writing database drivers for Yandex's userver asynchronous C++ framework used across the company's high-load infrastructure.

GPU kernel benchmarks | [KernelBench](#)

- Collaborating with Elliot Arledge on single-fused GPU kernel benchmarks measuring hand-optimized fused kernels.

TON blockchain | [ton.org](#)

- Currently studying the TON blockchain — its virtual machine, smart-contract model and network architecture.

Plazma | [GitHub](#)

- Native Qt6 / QML desktop video client in modern C++20: Telegram-account authentication via TDLib and hardware-accelerated libmpv playback (VA-API, NVDEC, software fallback), backed by a self-hosted media server with MinIO object storage.

napi-kv | [GitHub](#)

- Bitcask-model, append-only key/value store for Node.js implemented in C++ and exposed to JavaScript through the Node-API (N-API) native addon interface.

Kurtex | [GitHub](#)

- Vitest-style testing framework written in Rust on top of SWC, rusty_v8 and Deno core low-level APIs — watch mode, dependency-test resolution, snapshot manager, logger and reporter.

sockd | [GitHub](#)

- Linux keylogger daemon in C++ built as an offensive-tooling / systems-internals research project.